

Module 3 Question Pool

OMSCS 6250 Computer Networks

Lesson 3: Intradomain Routing

Routing Algorithms

Q1. [MCQ]

A packet arrives at a router inside an ISP. The router looks up the destination address of the packet in a table. The router then sends the packet out on one outgoing link.

How do forwarding and routing relate?

- **A.** Forwarding moves each packet from an input link to an output link; routing computes the table it uses.
- **B.** Forwarding runs in the control plane to build the table; routing runs in the data plane on each packet.
- **C.** Forwarding and routing are two names for one operation: looking up a packet's destination in the table.
- **D.** Forwarding fills the table from the packets that arrive; routing then shares that table with other routers.

Correct answer: **A**

Why: Forwarding happens inside one router, on each packet: the router looks up the destination in its forwarding table and moves the packet to an output link, in nanoseconds, in the data plane. Routing is the network-wide computation, in the control plane and over seconds, that fills the forwarding table.

Q2. [TF]

A network is modeled as a graph. Each router is a node. Each link between two routers is an edge, and each edge has a cost. The routers run OSPF, a link-state protocol, or RIP, a distance vector protocol.

The routing algorithm sets the cost of each edge to reflect how congested the link is at the moment.

- True
- False

Correct answer: **False**

Why: The routing algorithms take the edge costs as given. Under OSPF an operator chooses them, and they may reflect a link's length, speed or money cost. RIP counts every link as 1. RIP, OSPF and BGP are load-insensitive: a link's cost does not rise when the link is congested.

Link-state Routing Algorithm

Q3. [MCQ]

Every router in an AS runs the link-state routing algorithm. Router u is about to run Dijkstra's algorithm, with router u as the source node.

What information must router u already have before the algorithm starts?

- **A.** A copy of every other router's forwarding table, received from each router whenever that table changes.
- **B.** The costs of its own attached links only, since Dijkstra learns each further link as it iterates.
- **C.** The cost of every link in the network, learned from the packet each node broadcasts about its links.
- **D.** The distance vector of each neighbor, as most recently received, holding its cost to every node.

Correct answer: **C**

Why: Link-state is the centralized algorithm: every node knows the whole topology and every link cost before it computes. Each node broadcasts a link-state packet describing its own attached links, and together these give every node the same complete map, on which Dijkstra then runs. Neighbors' distance vectors belong to the distance vector algorithm, and forwarding tables are the output each router computes.

Q4. [MCQ]

Router u runs the link-state routing algorithm. Router u holds the complete map of the AS and runs Dijkstra's algorithm on this map, with router u as the source node. The algorithm has just terminated.

What does router u know at this point?

- **A.** u knows the full topology, but the next hops need a second run of Dijkstra from each neighbor.
- **B.** u knows least-cost paths to its direct neighbors, and computes longer ones when a packet needs them.
- **C.** u knows the least cost to every other node, but not the next hop, since Dijkstra keeps no predecessor.
- **D.** u knows the least cost to every other node and, by chaining predecessors, the next hop toward each.

Correct answer: **D**

Why: Dijkstra returns the least cost from u to every other node. It also leaves a predecessor $p(v)$ for each node. Chaining the predecessors back to u gives the whole path, and its first hop is the next hop stored in the forwarding table. This one run from u gives every next hop, so no run from a neighbor is needed. All paths are computed in the control plane before packets arrive, not when a packet needs one.

Q5. [TF]

Router u runs Dijkstra's algorithm, with router u as the source node. Node v is another node in the network, and node v is not directly attached to router u . $D(v)$ denotes the cost of the current least-cost path from u to v .

In the initialization step, the algorithm sets $D(v)$ to infinity.

- True
- False

Correct answer: **True**

Why: In the initialization step, each direct neighbor of u starts at the cost of its link from u . Every node not attached to u , such as v , starts at infinity, since no path to it is known yet. N' starts with u alone.

Linkstate Routing Algorithm - Example

Q6. [MCQ]

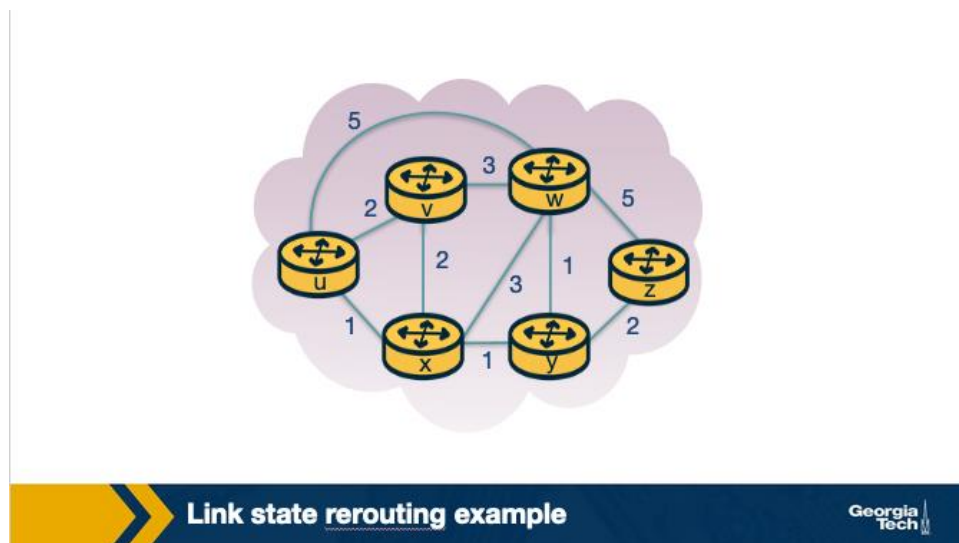


Figure: Link-state example topology (source = u) (Module 3)

Consider a network of six routers, u , v , w , x , y and z . Router u runs Dijkstra's algorithm, with router u as the source node. After the initialization step, $N' = \{u\}$, $D(v) = 2$, $D(w) = 5$ and $D(x) = 1$. $D(y)$ and $D(z)$ are infinity. In iteration 1, the algorithm adds node x to N' .

Why does the algorithm add node x to N' in iteration 1?

- A. Because x has the lowest current cost D of all the nodes not yet in N' .
- B. Because Dijkstra adds the direct neighbors of u first, in order of link cost.
- C. Because adding x first lets the algorithm finish in fewer iterations.
- D. Because x is the first neighbor of u to advertise its distance vector to u .

Correct answer: **A**

Why: Each iteration adds the node outside N' with the least current cost D . After initialization that is x , at 1. Neighbors of u get no priority: in this example y , which is not a neighbor of u , joins N' before w , which is. Each iteration adds exactly one node, so the order leaves the number of iterations unchanged. Router u holds the full map and computes D itself; advertised distance vectors belong to distance vector routing.

Q7. [MCQ]

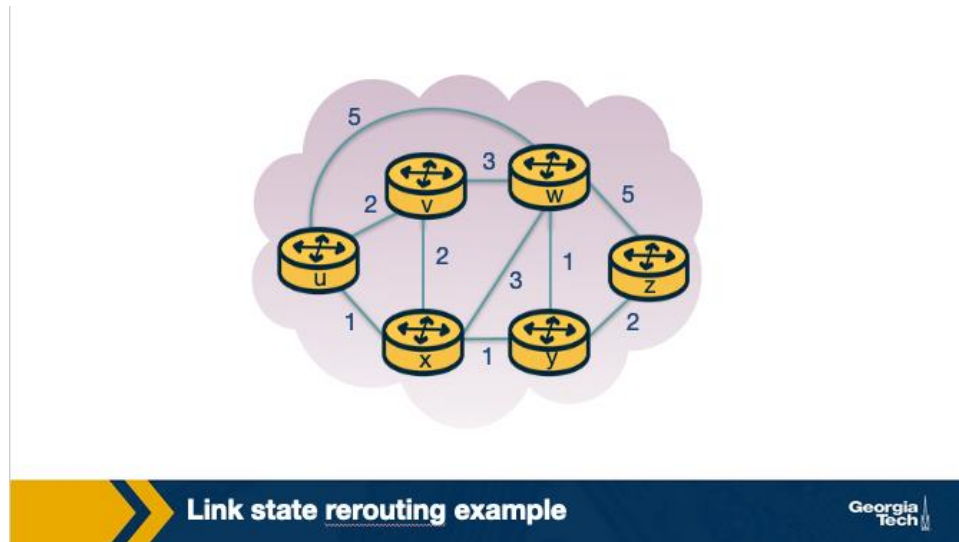


Figure: Link-state example topology (Module 3)

Consider a network of six routers, u, v, w, x, y and z. Each link has the same cost in both directions. Router u runs Dijkstra's algorithm, with router u as the source node. After the initialization step, $N' = \{u\}$, $D(v) = 2$, $D(w) = 5$ and $D(x) = 1$. In iteration 1, the algorithm adds node x to N' . $p(w)$ denotes the previous node along the current least-cost path from u to w.

What are the values of $D(w)$ and $p(w)$ at the end of iteration 1?

- A. $D(w) = 5$ and $p(w) = u$
- B. $D(w) = 4$ and $p(w) = x$
- C. $D(w) = 3$ and $p(w) = y$
- D. $D(w) = 4$ and $p(w) = u$

Correct answer: **B**

Why: Iteration 1 adds x, with $D(x) = 1$, and updates each neighbor of x. For w, the new cost is the smaller of the old $D(w)$, 5, and $D(x) + c(x, w) = 1 + 3 = 4$. $D(w)$ becomes 4, and $p(w)$ becomes x, the last node before w on the new path; the cost and the predecessor change together. $D(w)$ reaches 3 through y later, once y has joined N' .

Q8. [MCQ]

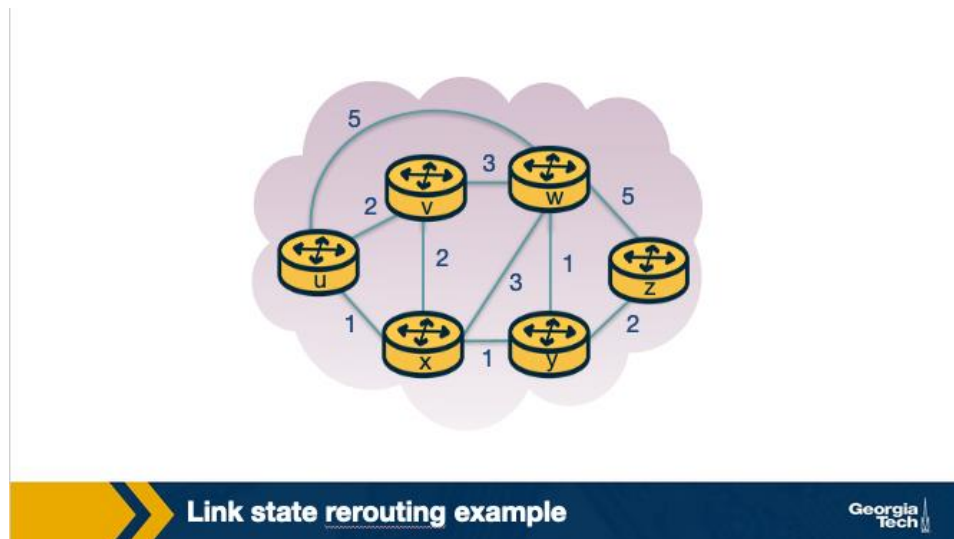


Figure: Link-state example topology (Module 3)

Consider a network of six routers, u, v, w, x, y and z. Router u runs Dijkstra's algorithm, with router u as the source node. Iteration 1 has added node x to N' . Nodes v and y are not in N' . $D(v) = 2$ and $D(y) = 2$. No other node outside N' has a lower cost. In iteration 2, the algorithm adds one node to N' .

How does the algorithm choose between node v and node y?

- A. By the higher router ID: the tied node with the larger router identifier joins N' first.
- B. By the last update: the tied node whose cost D was changed most recently is added to N' first.
- C. By no fixed rule: the algorithm adds either one of the two tied nodes, v or y, to N' first.
- D. By the direct link from u: a tied node that is a direct neighbor of u is added to N' first.

Correct answer: C

Why: The algorithm breaks such a tie arbitrarily, and either choice produces correct least-cost paths. Router IDs play no part in the algorithm. The order in which the costs were last updated does not matter either. Nothing favors direct neighbors of u: the algorithm compares only the costs D , and v, a neighbor of u, and y, which is not, both have $D = 2$.

Linkstate Routing Algorithm - Computational Complexity

Q9. [MCQ]

A link-state router runs Dijkstra's algorithm on a network of n routers. The router keeps the nodes that are not yet in N' in an unsorted list. In each iteration, the router searches this list for the node with the least cost.

What is the computational complexity of these searches, summed over all the iterations?

- A. $O(n \log n)$, because each of the n searches for the least-cost node takes logarithmic time.
- B. $O(n)$, because the algorithm adds exactly one node to N' in each of its n iterations.
- C. $O(n^2 \log n)$, because each of the n searches sorts the list before it takes the least cost.
- D. $O(n^2)$, because the searches cover n nodes, then $n-1$, and so on down to a single node.

Correct answer: **D**

Why: Each iteration searches the unsettled nodes for the least cost. The first search covers n nodes, the next $n-1$, and so on, $n(n+1)/2$ in total, which is $O(n^2)$. A heap finds the minimum in logarithmic time, but this router searches an unsorted list. One pass through a list finds its least cost, so no search needs to sort the list. Counting only the iterations gives n and leaves out the search inside each.

Distance Vector Routing

Q10. [MCQ]

Node x runs the distance vector algorithm. For each neighbor v , node x knows the link cost $c(x,v)$. Node x also stores the distance vector D_v that node x most recently received from each neighbor v . $D_x(y)$ denotes the estimate of node x for the least cost from x to destination y .

Which equation does node x use to update $D_x(y)$?

- **A.** $D_x(y) = \min \text{ over neighbors } v \text{ of } \{ c(x,v) + D_v(y) \}$
- **B.** $D_x(y) = \min \text{ over neighbors } v \text{ of } \{ D_v(y) \}$, plus $c(x,v)$ for that v
- **C.** $D_x(y) = c(x,v) + D_v(y)$, for the neighbor v with the cheapest link
- **D.** $D_x(y) = \min \text{ over all nodes } v \text{ of } \{ c(x,v) + D_v(y) \}$

Correct answer: **A**

Why: The Bellman-Ford equation takes, for each neighbor v , the link cost to v plus v 's advertised cost to y , and keeps the minimum. Picking the neighbor with the smallest $D_v(y)$ first can miss a cheaper total through a neighbor with a cheaper link. Picking the cheapest link first makes the opposite mistake. A node hears only from its neighbors, so the minimum runs over its neighbors.

Q11. [MCQ]

In the distance vector algorithm, each node keeps its own distance vector. Each node sends its distance vector to other nodes.

To which nodes does a node send its distance vector?

- **A.** To every node in the AS, by flooding it the way link-state packets are sent.
- **B.** Only to its directly attached neighbors, over the links it shares with them.
- **C.** Only to one central node, which collects the vectors and computes the paths.
- **D.** Only to the neighbor it routes through, for each destination in the vector.

Correct answer: **B**

Why: A distance vector node sends its vector to its directly attached neighbors, and they use it to update their own. Flooding to every node is how link-state packets travel. No central node is involved; the computation is distributed. Poison reverse changes what a node tells the neighbor it routes through, not which neighbors receive the vector.

Q12. [TF]

In the distance vector algorithm, each node receives distance vectors from its directly attached neighbors. Each node uses the distance vectors it receives to update its own distance vector.

Each node waits until every neighbor has sent a new distance vector, and only then recomputes its own distance vector.

- True
- False

Correct answer: **False**

Why: The algorithm is asynchronous. It does not require the nodes to operate in lockstep, and a node recomputes whenever a neighbor's new vector arrives.

Distance Vector Routing Example

Q13. [MCQ]

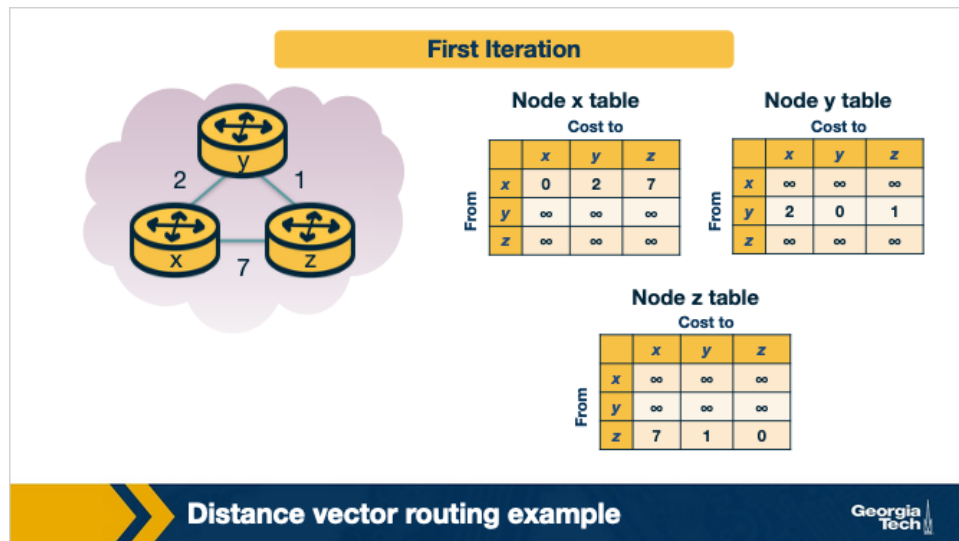


Figure: DV example, three-node topology x, y, z (Module 3)

Consider a network of three nodes, x, y and z. The nodes run the distance vector algorithm. The link costs are $c(x,y) = 2$, $c(y,z) = 1$ and $c(x,z) = 7$. In the second iteration, node x has received the first distance vectors of node y and node z. Node x applies the Bellman-Ford equation for destination z.

What are the cost $D_x(z)$ and the next hop from node x to destination z after this computation?

- A. $D_x(z) = 3$, with next hop z, since z is the destination.
- B. $D_x(z) = 7$, with next hop z, since direct links win early.
- C. $D_x(z) = 3$, with next hop y, from $2 + D_y(z) = 2 + 1$.
- D. $D_x(z) = 8$, with next hop y, from $c(x,z) + D_y(z) = 7 + 1$.

Correct answer: **C**

Why: x compares its two neighbors. Through y the cost is $c(x,y) + D_y(z) = 2 + 1 = 3$; through z it is $c(x,z) + D_z(z) = 7 + 0 = 7$. The minimum is 3, and the neighbor that achieves it, y , is the next hop, not the destination z . A direct link wins only when it is cheapest. Adding $c(x,z)$ to $D_y(z)$ pairs the link to one neighbor with the estimate of another.

Q14. [MCQ]

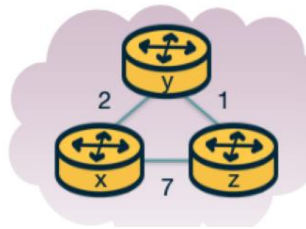


Figure: DV example, three-node topology (Module 3)

Consider a network of three nodes, x , y and z . The nodes run the distance vector algorithm. In the first iteration, node x had the distance vector $(0, 2, 7)$, node y had $(2, 0, 1)$, and node z had $(7, 1, 0)$. Each distance vector lists the costs to x , y and z , in that order. In the second iteration, each node received the distance vectors of its neighbors and recomputed its own distance vector.

Which nodes send their distance vectors in the third iteration?

- A. x and z send, because their vectors changed; y stays silent.
- B. x , y and z send, because each node sends once per iteration.
- C. Only x sends, because only x found a cheaper path to a destination.
- D. No node sends, because the nodes already hold their final least costs.

Correct answer: **A**

Why: A node sends an update only if its own distance vector changed. In the second iteration, x lowered its cost to z from 7 to 3, through y at $2 + 1$. z lowered its cost to x from 7 to 3, through y at $1 + 2$. y 's own vector, $(2, 0, 1)$, did not change, so y stays silent. z also found a cheaper path, so x is not alone. Each node's own costs are already final, but y still holds the old vectors of x and z , so the exchange is not over.

Q15. [TF]

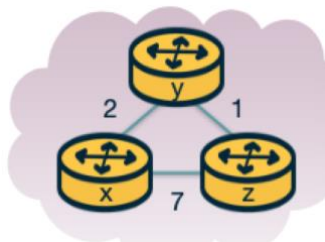


Figure: DV example, three-node topology (Module 3)

Consider a network of three nodes, x , y and z . The nodes run the distance vector algorithm. The algorithm has converged. Every node has received the current distance vector of each neighbor. No link cost changes after this point.

The nodes send no further distance vectors to each other.

- True
- False

Correct answer: **True**

Why: A node sends its vector only when that vector changes. Once no vector changes, nothing is sent, and the nodes enter a quiescent state. They stay quiet until a link cost changes.

Q16. [MCQ]

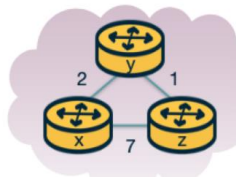


Figure: DV example, three-node topology (Module 3)

Consider a network of three nodes, x, y and z. The nodes run the distance vector algorithm. The link costs are $c(x,y) = 2$, $c(y,z) = 1$ and $c(x,z) = 7$. The algorithm has converged. Then link x-z fails. Links x-y and y-z stay up.

After the algorithm converges again, what are the cost $D_x(z)$ and the next hop from node x to destination z?

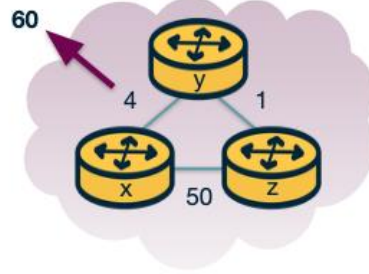
- A. $D_x(z) = 3$, via y: $c(x,y) = 2$ plus $D_y(z) = 1$, from y's vector.
- B. $D_x(z) = \text{infinity}$: x's own route to z used the failed link.
- C. $D_x(z) = 2$, via y: the cost of the link to y, the next hop to z.
- D. $D_x(z) = 1$, via y: y's cost to z, copied into x's table.

Correct answer: **A**

Why: Before the failure, x already reached z through y, at $2 + 1 = 3$, so the failed link was not on its path. $D_x(z)$ stays 3, via y. y reaches z over its own direct link at cost 1, so y never used the x-z link either. x's estimate adds the link cost to the neighbor, $c(x,y) = 2$, to what that neighbor reports, $D_y(z) = 1$. Taking the link cost alone, or the neighbor's cost alone, leaves out one of the two terms.

Link Cost Changes and Failures in DV - Count to Infinity Problem

Q17. [MCQ]



Link cost changes and Failures in DV

Georgia Tech

Figure: Count-to-infinity, link x-y rises to 60 (Module 3)

Consider a network of three nodes, x, y and z. The nodes run the distance vector algorithm. The link costs are $c(x,y) = 4$, $c(y,z) = 1$ and $c(x,z) = 50$. The algorithm has converged. Then the cost of link x-y increases to 60. Node y detects the change and computes a new cost to x of $1 + 5 = 6$.

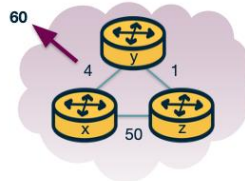
Where does the value 5 in this computation come from?

- A. From z's advertised cost to x, which y takes to be z's direct link to x.
- B. From z's advertised cost to x, a stale value that z computed through y itself.
- C. From x's advertised cost to z, $4 + 1$, which x sent to y before the change.
- D. From z's new cost to x, since z learns of the change before y recomputes.

Correct answer: **B**

Why: y applies the Bellman-Ford equation. Through z its cost is $c(y,z) + Dz(x) = 1 + 5 = 6$, which beats the direct link at 60. The 5 is the cost z advertised before the change, which z computed through y, as $1 + 4$, while z's direct link costs 50. y cannot tell that z's path runs back through itself, and z has not yet heard of the change. y's cost to x uses its neighbors' costs to x, so x's cost to z is outside the equation.

Q18. [MCQ]



Link cost changes and Failures in DV

Georgia Tech

Figure: Count-to-infinity, the slow climb (Module 3)

Consider a network of three nodes, x, y and z. The nodes run the distance vector algorithm. The algorithm has converged. Then the cost of link x-y increases from 4 to 60. Node y now routes to x through node z, and node z routes to x through node y. The cost estimates to x rise one unit at a time: node y computes 6, then node z computes 7, then node y computes 8. After 44 iterations, node z switches to the direct link x-z, at cost 50, and node y settles at a cost of 51.

Why does each new estimate exceed the previous one by exactly 1?

- **A.** Each node raises its estimate by one hop per exchange, since distance vector counts hops as cost.
- **B.** Distance vector limits each update to a change of one cost unit, whatever the real change was.
- **C.** Each node adds the y-z link cost, 1, to the other's last estimate, which ran back through itself.
- **D.** The nodes detect the routing loop, and raise their costs in small steps to avoid oscillation.

Correct answer: **C**

Why: Each node applies the Bellman-Ford equation to the other's last advertised cost. y adds $c(y,z) = 1$ to z's estimate, and z adds 1 to y's, but each estimate was computed through the other node, so the estimate climbs by the y-z link cost, 1, per step. The metric is link cost, and nothing caps the size of an update. The algorithm never detects the loop; the climb continues until z's direct link at 50 is cheaper.

Q19. [TF]

Consider a network of three nodes, x, y and z. The link costs are $c(x,y) = 4$, $c(y,z) = 1$ and $c(x,z) = 50$. The nodes run the distance vector algorithm. The algorithm has converged.

The distance vectors settle in fewer iterations after the cost of link x-y drops to 1 than after it rises to 60.

- True
- False

Correct answer: **True**

Why: Good news travels fast, and bad news travels slowly. When the cost of link x-y drops to 1, the new cost beats the existing estimates at once, and the distance vectors settle in two iterations. When the cost rises to 60, y builds a new route to x on z's stale cost of 5, which runs back through y itself. The estimates then climb one unit per exchange, which is the count-to-infinity problem. z switches to its direct link, at cost 50, only after 44 iterations.

Poison Reverse

Q20. [MCQ]

Consider a network of three nodes, x, y and z, with a link between each pair of nodes. The nodes run the distance vector algorithm with poison reverse. The link costs are $c(x,y) = 4$, $c(y,z) = 1$ and $c(x,z) = 50$. Node z reaches node x through node y, at a cost of 5.

What does node z advertise to node y about the cost from z to x?

- **A.** z advertises $D_z(x) = \text{infinity}$ to y, so y never routes to x through z.

- B. z advertises its true cost, 5, and y discards it, since y sees z routes via y.
- C. z omits x from the vector it sends y, so y has no entry for x through z.
- D. z advertises infinity for x to all its neighbors, not only to y.

Correct answer: **A**

Why: Under poison reverse, z tells y that its distance to x is infinity, because z routes to x through y. y then never routes to x through z, and cannot reuse its own stale cost. y has no way to see that z's path runs back through y, so z must report infinity explicitly. The entry stays in the vector with the value infinity, and only y, the neighbor z routes through, receives it.

Q21. [MCQ]

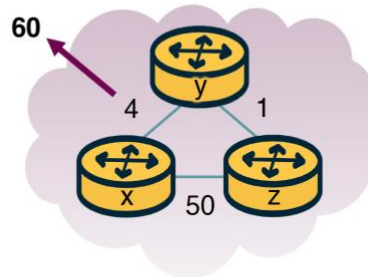


Figure: Poison reverse, after the change (Module 3)

Consider a network of three nodes, x, y and z. The nodes run the distance vector algorithm with poison reverse. The link costs are $c(x,y) = 4$, $c(y,z) = 1$ and $c(x,z) = 50$. Node z reaches node x through node y. Node z advertises to node y a cost to x of infinity. Then the cost of link x-y increases to 60. Node z now routes to x over the direct link x-z, at cost 50.

What does node z now advertise to node y about the cost from z to x?

- A. z keeps advertising infinity, since the poison lifts only when link x-y recovers.
- B. z advertises its real cost to x, since the lie lasts only while z routes via y.
- C. z omits x from the vector it sends y, since y is no longer on z's path to x.
- D. z advertises 61, the cost of its old route to x through y, until y updates.

Correct answer: **B**

Why: z advertises infinity only while it reaches x via y. Once z routes over its direct link, it tells y its real cost, $D_z(x) = 50$, and y settles at 51 through z. The poison follows z's current route, whatever state link x-y is in. z still sends an entry for x, and its value is the cost of the route z now uses.

Q22. [TF]

A network of routers runs the distance vector algorithm with poison reverse.

Poison reverse fully eliminates the count-to-infinity problem in all topologies, including routing loops that involve three or more nodes that are not all directly connected.

- True
- False

Correct answer: **False**

Why: Poison reverse solves the two-node loop, because a node lies only to the neighbor it routes through. In a loop of three or more nodes, no node lies to the node whose stale cost it will reuse. The loop still forms, and the estimates still climb slowly.

Linkstate Routing Protocol Example: OSPF

Q23. [MCQ]

An AS runs RIP, a distance vector routing protocol. The network operator plans to replace RIP with OSPF.

Which statement describes how OSPF operates?

- **A.** OSPF is distance vector like RIP, but it allows paths far longer than RIP's cap of 15 hops.
- **B.** OSPF is link-state, and like RIP it runs over UDP and relies on it to deliver updates.
- **C.** OSPF sends each router's full distance vector to its neighbors, but only every 30 minutes.
- **D.** OSPF is link-state: each router floods its link states in its area and runs Dijkstra itself.

Correct answer: **D**

Why: OSPF is a link-state protocol. Each router floods link-state advertisements to the other routers in its area, builds the same map, and runs Dijkstra on it with itself as the root. OSPF runs directly on IP and provides its own reliable delivery, while RIP runs over UDP. The 30 minutes is OSPF's refresh period for its link-state advertisements; distance vectors belong to RIP.

Q24. [MCQ]

A network designer proposes that all the routers on the Internet form one flat network and run one routing algorithm. This design fails for two reasons. Grouping the routers into autonomous systems solves both problems.

What are the two reasons the flat design fails?

- **A.** Scale, as state and updates grow too large, and each organization's need to run its own network.
- **B.** Scale, as state and updates grow too large, and the need to authenticate the routing updates.
- **C.** The lack of one address format across routers, and each organization's need to run its own network.
- **D.** Security, as routing updates go unauthenticated, and the lack of one address format across routers.

Correct answer: **A**

Why: The two reasons are scale and administrative autonomy. Hundreds of millions of routers would need enormous tables, and the updates among them would be huge. An organization also wants to choose its own routing algorithm and to hide its internal structure. A shared address format is not one of the two reasons. Authentication is an OSPF feature an operator configures, not a reason for autonomous systems.

Q25. [TF]

An AS runs OSPF, and the AS is divided into areas. Exactly one area is the backbone area. Router r1 is inside area 1, and router r2 is inside area 2. Neither area 1 nor area 2 is the backbone area, and neither router is an area border router.

A packet from router r1 to router r2 goes from an area border router of area 1 directly into area 2, without passing through the backbone area.

- True
- False

Correct answer: **False**

Why: Routing between two areas follows a fixed path. The packet goes to an area border router of area 1, then through the backbone, then to the area border router of area 2, and only then to router r2. The main role of the backbone area is to route traffic between the other areas, and the backbone contains every area border router in the AS.

Hot Potato Routing

Q26. [MCQ]

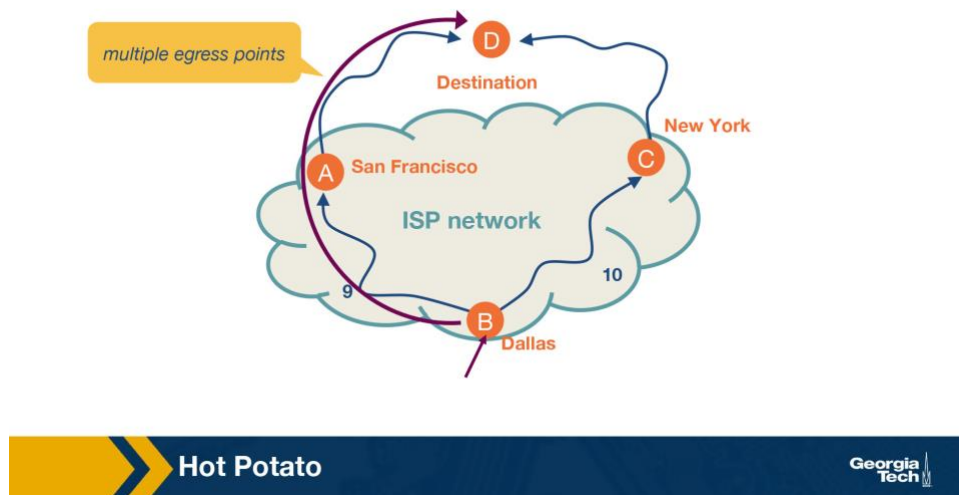


Figure: Hot potato routing scenario (Module 3)

Consider an ISP network with a router in Dallas. The router in Dallas forwards traffic toward destination D. The ISP has two egress points toward D, one in San Francisco and one in New York. Outside the ISP, the path from New York to D is shorter than the path from San Francisco to D. The BGP routes through the

two egress points are equally good. The IGP path cost from Dallas is 9 to San Francisco and 10 to New York. The router in Dallas uses hot potato routing.

Which egress point does the router in Dallas choose?

- **A.** Either one at random: equally good BGP routes are split evenly across both egresses.
- **B.** New York: the higher IGP cost egress is chosen, to spread load across the network.
- **C.** San Francisco: among equally good BGP routes, the lower IGP cost to the egress wins.
- **D.** New York: hot potato picks the egress nearer to D, to shorten the path outside the ISP.

Correct answer: **C**

Why: Hot potato routing picks the closest egress point, measured by IGP path cost, when the BGP routes are otherwise equally good. San Francisco, at 9, is closer than New York, at 10. The choice uses only the cost inside the ISP, so the length of the path beyond the egress points is ignored, and the lower cost wins because it gets traffic out of the network sooner. Hot potato is a fixed tie break; only the tie break after it is vendor-dependent.

Q27. [MCQ]

Consider an ISP network. The ISP has several egress points toward the same destination. The BGP routes through these egress points are equally good. Each router in the network applies hot potato routing independently.

How do the routers in the network choose the egress point for traffic toward the destination?

- **A.** Each router sends traffic to the egress with the most spare capacity at that moment.
- **B.** Each router picks the egress with the shortest AS path, so internal costs play no part.
- **C.** Each router picks the egress closest to where the traffic entered the AS, not to itself.
- **D.** Each router sends traffic to its closest egress, and the next router picks the same one.

Correct answer: **D**

Why: Each router sends the traffic toward its own closest egress point, by IGP path cost measured from that router. The next router on the path makes the same choice and picks the same egress, so the path stays consistent. IGP path costs stay fixed as load changes, and AS path length is an earlier step in the BGP decision process that has already tied.

Q28. [MCQ]

A network uses hot potato routing. The network hands traffic for an outside destination to a neighboring network.

How does hot potato routing affect the resource consumption of the two networks?

- **A.** The neighboring network reduces its own resource consumption.
- **B.** The handing-off network reduces its own resource consumption.
- **C.** Both networks reduce their resource consumption by the same amount.
- **D.** Resource consumption stays the same in both of the networks.

Correct answer: **B**

Why: Hot potato routing reduces the resource consumption of the network that picks the closest egress point, because that network gets the traffic out early. The neighboring network then takes over carrying the traffic, so the saving stays with the network that hands it off. The saving is real: the IGP costs to the egress points differ, and choosing the lower one is the saving.

Q29. [MCQ]

A network uses hot potato routing. The network hands traffic for an outside destination to a neighboring network.

Why does hot potato routing reduce the resource consumption of the network that hands off the traffic?

- **A.** The traffic follows the shortest end-to-end path, so each network carries it a shorter distance.
- **B.** The egress point with the lowest IGP cost also has the most spare capacity toward the neighbor.
- **C.** The traffic exits the network early, and the neighbor then carries it the rest of the way.
- **D.** The routers skip the BGP decision process and pick the egress point from IGP costs alone.

Correct answer: **C**

Why: Handing the traffic off at the closest egress point gets it out of the network early, and the traffic then becomes the neighboring network's cost to carry, which gives hot potato its name. Hot potato minimizes the handing-off network's internal cost, so the traffic need not follow the shortest path overall. IGP cost measures the internal path to the egress point, whatever the spare capacity on the link to the neighbor. Hot potato is a tie break inside the BGP decision process, used only after the earlier criteria have tied.

Q30. [TF]

An ISP uses hot potato routing. An operator changes the cost of one link inside the ISP. After the change, a different egress point is the closest one for many destinations. The BGP routes through the old egress point are still available.

Neighboring networks receive BGP updates caused by the link cost change inside the ISP.

- True
- False

Correct answer: **True**

Why: Hot potato routing ties the choice of egress point to internal path costs. When an internal cost change makes a different egress point closer, the routers switch, even though the old BGP route still works. The switch changes the BGP routes the ISP uses, so neighboring networks see BGP updates. An internal event has become an external one, which breaks the isolation that the two-tier routing architecture is meant to give.